

### 8.1. Serial and Parallel Communication Interfaces

Peripherals connected to a computer need special communication links for interfacing them with the CPU. The purpose of those communication links are to resolve the differences that exist between the processor and each peripherals.

The data transfer rate of peripherals is usually slower than the transfer rate of CPU, and consequently synchronization mechanism may be needed.

The operating modes of peripherals are different from each other and each must be controlled so as not to disturb the operation of other peripherals connected to CPU. Data codes and formats of peripherals differ from that of CPU and memory.

To resolve these differences, computer systems include special hardware components between the CPU and peripherals, to supervise and synchronize all inputs and outputs transfer. These components are called interface units, because they interface between the processor bus and peripheral devices.

The two major types of I/O interfaces are:

a) Serial Interface

b) Parallel Interface

a) Serial Interfacing:

↳ Exchange data with the peripherals in serial mode.

↳ data are transmitted one bit at a time.

↳ slow, inexpensive to implement

↳ Serial interface converts parallel mode bus systems to serial mode

↳ If the bus has 'n' data lines, the serial I/O interface accepts 'n' bits of data simultaneously from the bus. These 'n' bits are sent to the I/O devices, one bit at a time, requiring 'n' time slots for transmission.

↳ E.g. Keyboard Interfacing:

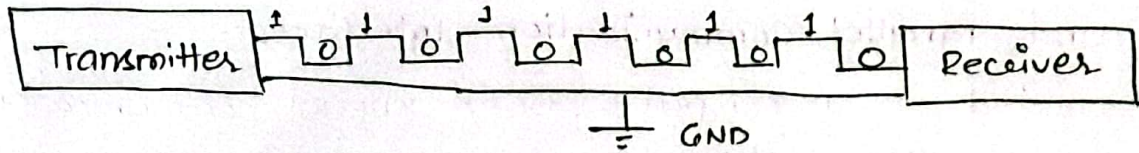


fig: serial data transfer.

- ↳ A parallel to serial converter is used to convert the incoming parallel data to serial form and then the data is sent out with the least significant Bit,  $D_0$  first and most significant Bit,  $D_7$  at last.
- ↳ The time taken to transmit a word in serial data transmission will be 'n' times more than the time taken in parallel data transmission.
- ↳ Types of serial data transfer:
  - i) Asynchronous serial data transfer
  - ii) Synchronous serial data transfer.

### ↳ Parallel Interfacing

Some I/O devices can handle data at speeds that cannot be supported by serial interfaces. In such case, parallel interface is required.

In parallel interface, 'n' bits of data are handled simultaneously by the bus and on the links to the device. Parallel Interfacing interchange data at faster speed and is expensive due to need of multiple wires.

Many I/O devices, particularly those requiring high data transfer rate use this arrangement.  
eg: Printer Interface.

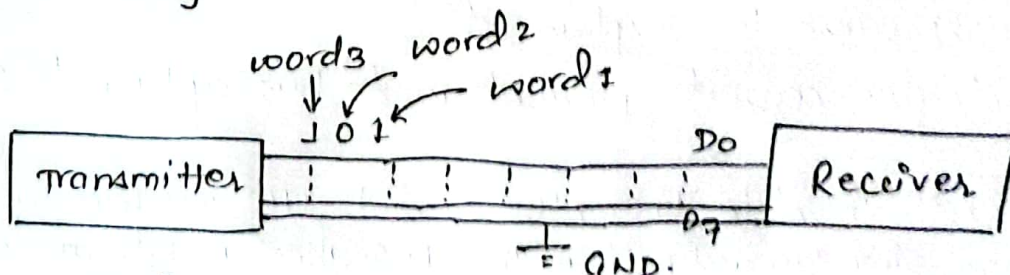


fig: Parallel data transfer.

- ↳ A word of 'n' bits is transmitted in parallel.
- ↳ channel comprises of n-lines, last line is called common ground.
- ↳ time required to transfer one word is equal to time taken for a bit to transmit.
- ↳ Impractical, for long distance transmission, since costs for large number of lines increases.

## 8.2. Programmed I/O :

Data transfer between CPU and I/O devices may be handled in variety of modes. It may be handled in one of the 3 possible modes.

- i) Programmed I/O
- ii) Interrupt driven I/O
- iii) Direct memory Access (DMA).

### Programmed I/O

In programmed I/O, the processor executes a program that gives it direct control of the I/O operation, including sensing I/O device status, sending a read or write command and transfer the data.

In programmed I/O, each data transfer is initiated by the instructions in the CPU and hence the CPU continuously monitors the interfaces.

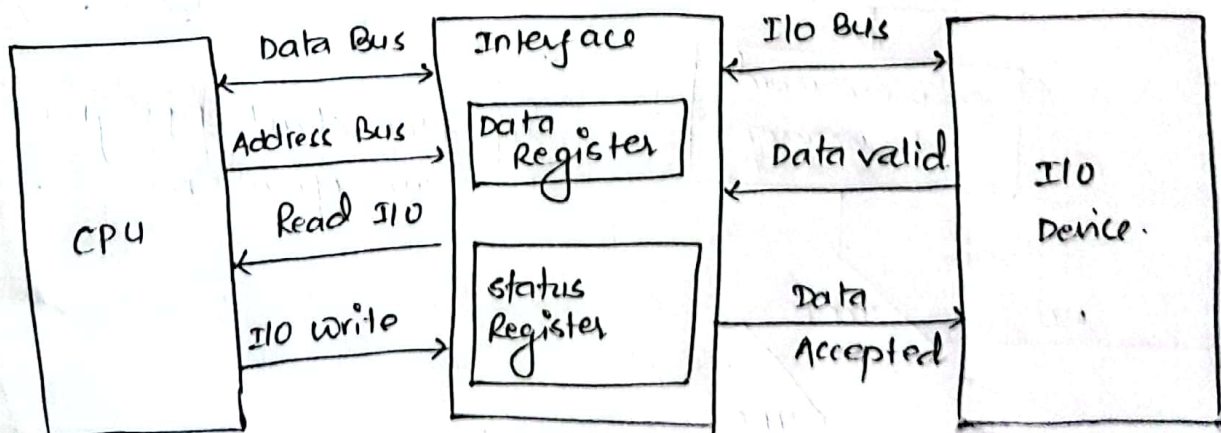


fig: Transfer of data from I/O to CPU

Input instruction is used to transfer data from I/O device to CPU, store instruction is used to transfer data from CPU to memory and output instruction is used to transfer data from CPU to I/O device.

Here, the CPU directly monitors the interface and controls data transactions between I/O device and CPU itself. I/O devices operate at much slower speed as compared to the CPU. Because of this reason, many wait states are imposed during data transaction and valuable time of processor is wasted. Thus this technique is generally used in slow computers and is not an efficient method if the speed of CPU and I/O is different.

The flow of programmed I/O technique is shown below:

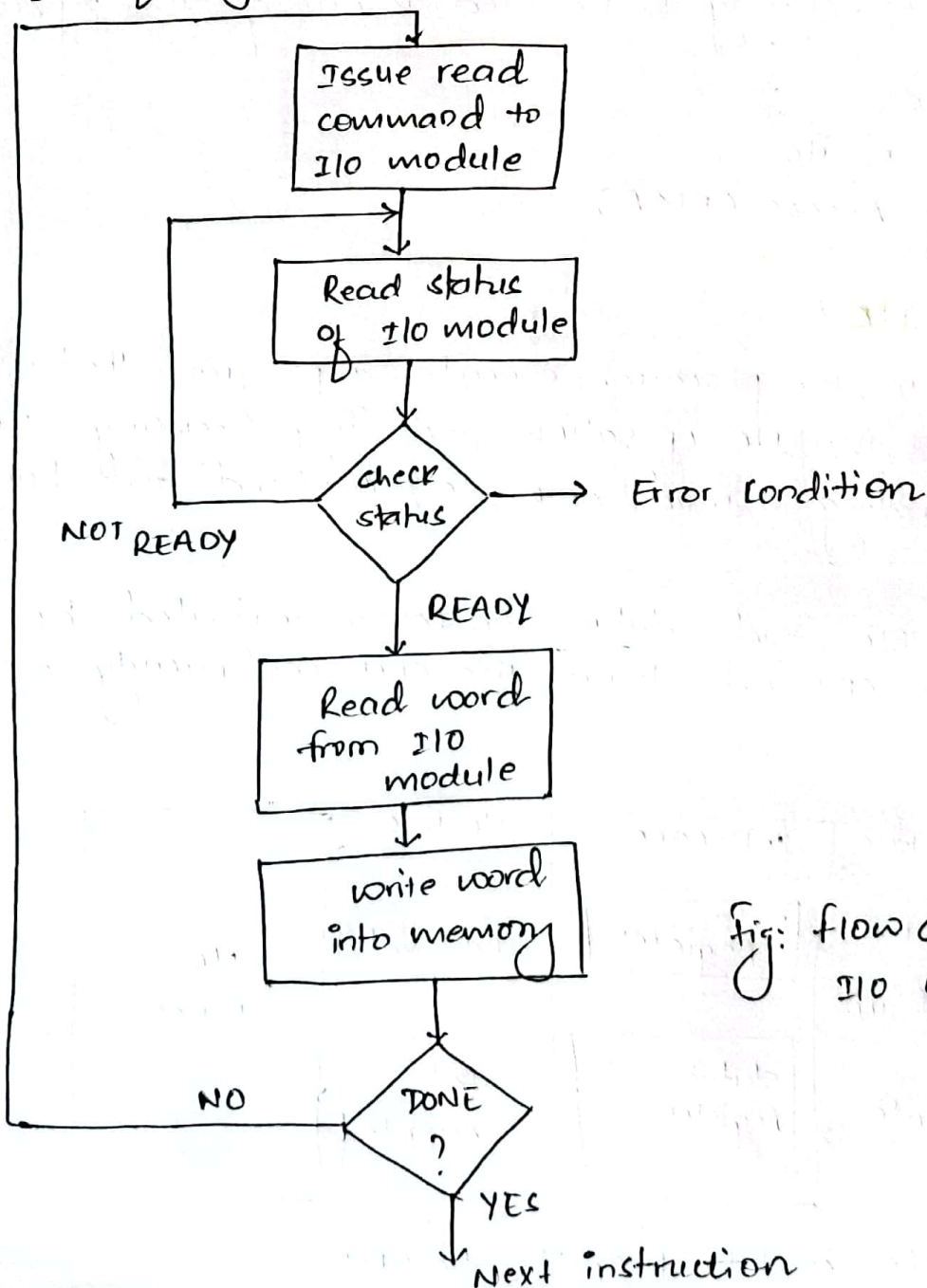


Fig: flow chart of programmed I/O operation.

### characteristics:

- ↳ continuously CPU involvement
- ↳ CPU slowed down to I/O speed
- ↳ simple
- ↳ least hardware

Polling refers to continuously sampling the status of an external I/O device. Hence programmed I/O is also known as polled I/O or software driven I/O.

### 8.3. Interrupts, Types of Interrupts, Interrupt Processing, Interrupt Hardware and Priority.

#### Interrupts:

- ↳ An interrupt is a signal from a device attached to a computer or from a program within the computer that requires the operating system to stop and figure out what to do next.
- ↳ Interrupt systems are nothing but while the CPU can process the programs if the CPU needs any I/O operation. Then it sends to the queue and it does the CPU process later.

#### Types of Interrupts:

There are two types of interrupts which are as follows-

##### 1. Hardware Interrupts

The interrupt signals generated from external devices and I/O devices are made interrupt to CPU when the instructions are ready.

for example -

In a keyboard if we press a key to do some action this pressing of the keyboard generates a signal that is given to the processor to do action. Such interrupts are called hardware interrupts.

- ↳ Hardware interrupts are classified into two types which are as follows -

- i. Maskable Interrupt
- ii. Non maskable Interrupt

↳ Maskable interrupts are those hardware interrupts that can be delayed when a highest priority interrupt has occurred to the processor.

↳ Non-Maskable interrupts are those interrupts that cannot be delayed and immediately be serviced by the processor.

## 2. Software Interrupts

The interrupt signal generated from internal devices and software programs need to access any system call then software interrupts are present.

Software interrupt is divided into two types. They are:

i) Normal Interrupt

ii) Exception

↳ Normal Interrupts are those interrupts that are caused by the software instructions. ~~are called software interrupt~~ system call, Timer Interrupt, Breakpoint interrupts, floating point interrupts lies with the normal interrupt type category.

↳ Exception is nothing but an unplanned interruption while executing a program. for eg. while executing a program if we got a value that is divided by zero is called an exception.

## # Interrupt Processing:

Interrupt processing refers to the mechanism by which a computer system handles interrupts. When an interrupt occurs, the normal execution of the processor is temporarily suspended, and the system transfers control to an interrupt handler or interrupt service routine (ISR) to handle the interrupt.

The interrupt processing typically involves the following steps:

1. Interrupt Detection
2. Context saving
3. Interrupt Acknowledgement
4. Interrupt Vectoring
5. ISR Execution
6. Interrupt servicing
7. Context Restore
8. Return from Interrupt.

Diagrammatic representation of interrupt processing is shown below.

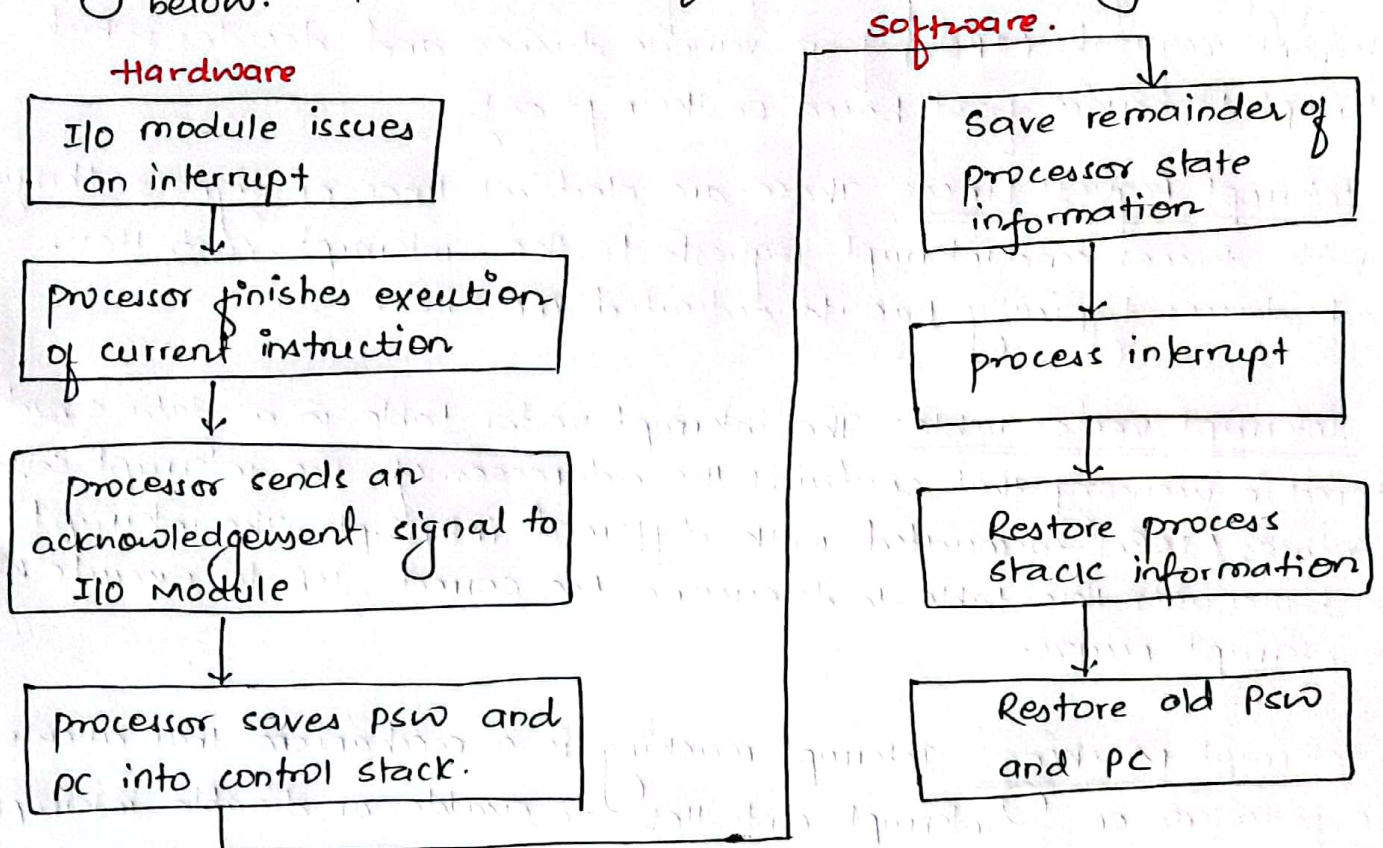


fig: simple interrupt processing.

## #. Interrupt Hardware and Priority:

Interrupt hardware and interrupt priority are two concepts related to how interrupts are managed and prioritized in computer system.

### 1. Interrupt Hardware:

Interrupt hardware refers to the physical circuitry and mechanisms within a computer system that enables the detection and handling of interrupts. It typically involves the following components:

#### i) Interrupt Controller:

An interrupt controller is a hardware component responsible for managing and prioritizing interrupts from multiple sources. It receives interrupt requests (IRQs) from various devices and decides which interrupt to service first based on their priority.

ii) Interrupt Request Lines: These are electrical lines or signals through which devices send interrupt requests to the interrupt controller. Each device typically has its dedicated IRQ lines.

iii) Interrupt vector Table: The interrupt vector table is a data structure stored in memory that contains the addresses of the interrupt service routines (ISRs) associated with different interrupts. The interrupt controller uses this table to determine the correct ISR to execute when an interrupt occurs.

iv) Interrupt Masking: Interrupt masking is a mechanism that allows the processor or interrupt controller to enable or disable interrupts selectively. By masking an interrupt, its corresponding IRQ line is ignored and the interrupt is not serviced.

### 2. Interrupt Priority:

Interrupt Priority determines the order in which interrupts are handled when multiple interrupts occur simultaneously or in rapid succession.

It ensures that the higher-priority interrupts are serviced before lower-priority ones.

- ↳ with multiple lines, processor itself picks the interrupt line with highest priority.
- ↳ with software polling and hardware polling the order in which I/O modules are polled determines the priority.
- ↳ with vectored method, priority can be determined by using bus arbiter.

#### 8.4. Direct Memory Access, I/O Processors.

##### #. DMA

DMA stands for Direct Memory Access. It is a technique used in computer systems to allow devices, such as disk controllers or network interfaces to transfer data directly to and from system memory without involving the CPU.

DMA improves system performance by offloading data transfer tasks from the CPU, which can then focus on other processing tasks.

In DMA, the MPU releases the control of the buses to a device called a DMA controller. The controller manages data transfer bet<sup>n</sup> memory and a peripheral under its control.

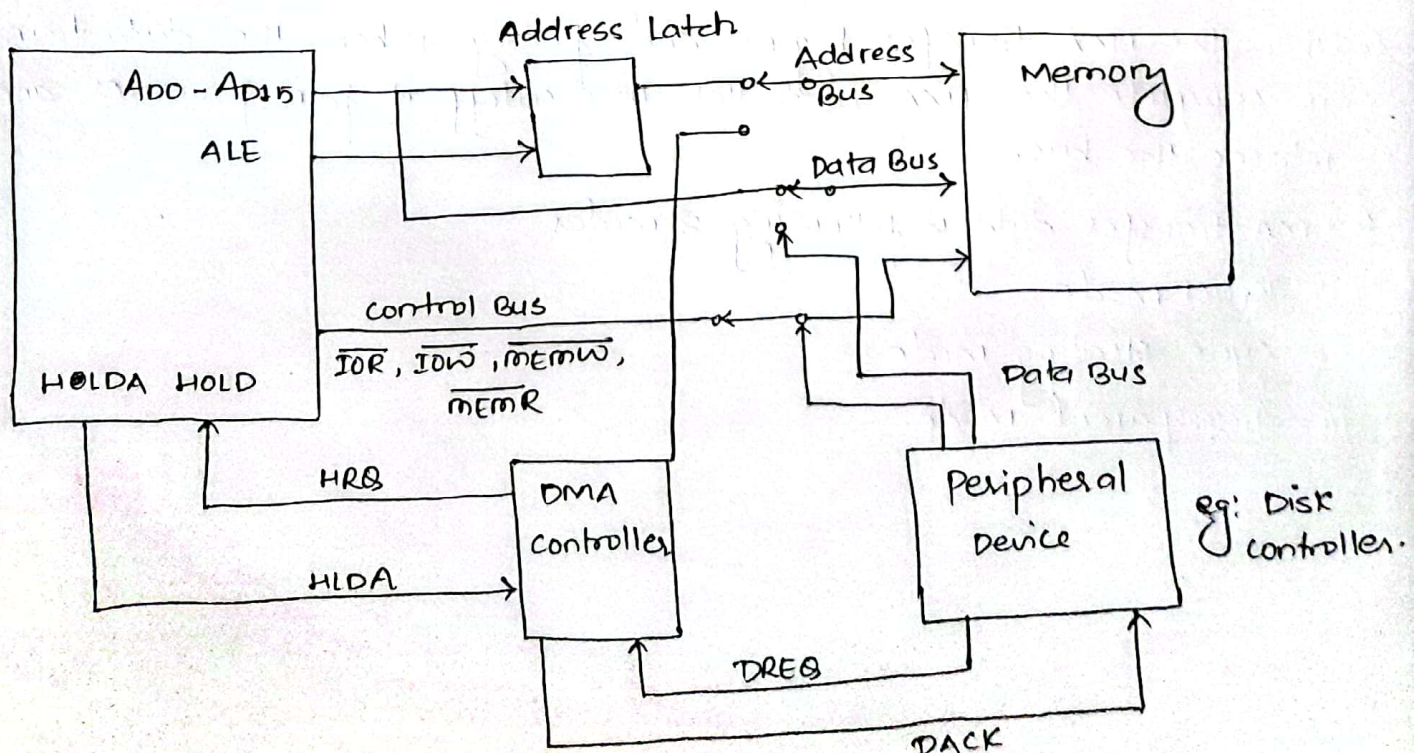


Fig: Block diagram showing DMA transfer.

The sequence of DMA transfer as shown in above block diagram can be explained below:

↳ Originally, microprocessor is connected to the memory as shown in fig above with switches closed for address, data and control bus.

When the peripheral wants to transfer data using DMA transfer it sends DMA request, DREQ signal to the DMA controller.

↳ If the I/P of the DMA controller is unmasked, the DMA controller will send a hold request HREQ signal to the  $\mu$ p's HOLD input.

↳ The  $\mu$ p finishes the current machine cycle and floats its buses, sending out a hold acknowledgement signal, HLDA to the DMA controller.

↳ When DMA controller receives HLDA signal it will send out a control signal which throws the 3 bus switches down to their DMA position.

This disconnects the processor from buses and connects DMA controller to the buses. Now, DMA controller sends out the address of the bytes to be transferred and sends out DACK signal to the peripheral device to tell it to get ready to output the byte.

↳ Then the DMA transfer begins and finally when the data transfer is complete, the DMA controller ~~cuts~~ cutoff it from processor and releases the bus.

↳ DMA transfer data is following 3 modes

i) Burst mode

ii) Cycle stealing mode

iii) Transparent mode.

## # I/O processor / I/O channel:

DMA controller improves the system performance by speeding up the transfer of data between I/O device and memory but it still leaves some work of CPU.

In some cases data from I/O device must be manipulated before storage. These shortcomings are overcome by using I/O processor or peripheral processing unit (PPU).

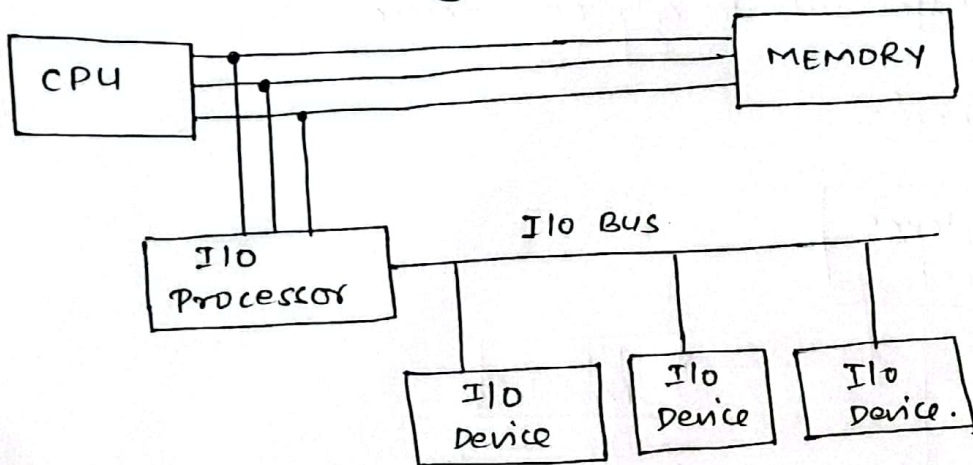


fig: system configuration of I/O processor.

### characteristics:

- ↳ I/O channel is a processor which has its own instruction set with I/O instructions and a local memory of its own.
- ↳ I/O channel accesses memory by cycle stealing.
- ↳ channel can execute channel program stored in memory called channel command word (CCW).
- ↳ CCW specifies parameters needed by the channel to control I/O device and perform data operations.
- ↳ CPU initiates the channel by executing I/O instruction and once initiated channel operates independently without CPU.

### I/O channel Architecture:

↳ two types:

- i) selector channel
- ii) multiplexer channel

### i) selector channel:

Data and address

path to  
memory

Control signal  
by CPU

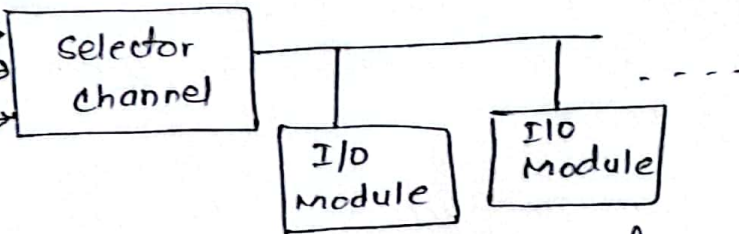


fig: selector channel Architecture.

- Selector channel controls multiple I/O devices.
- At one time, it transfer data to only one device.

### ii) Multiplexer channel:

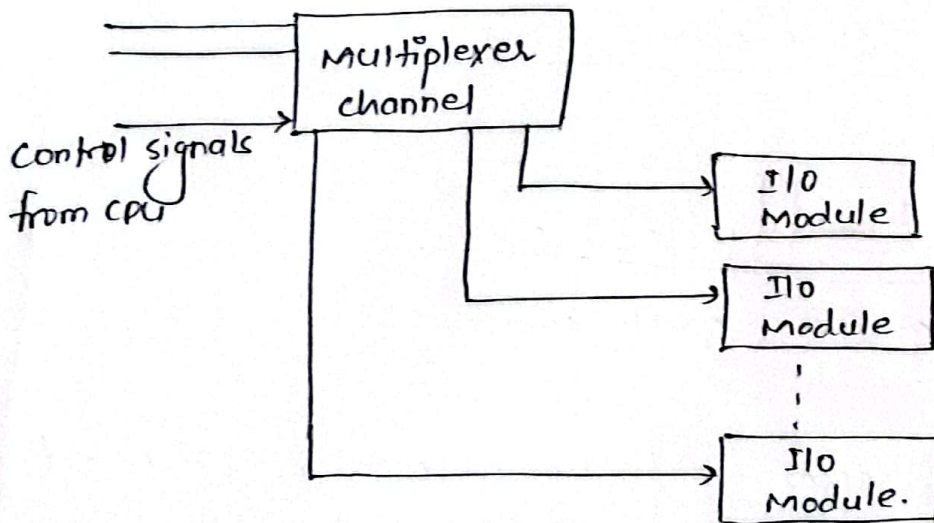
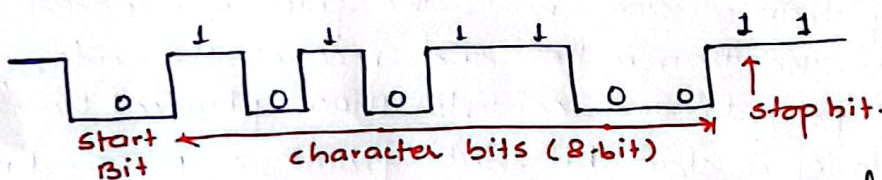


fig: Multiplexer channel architecture

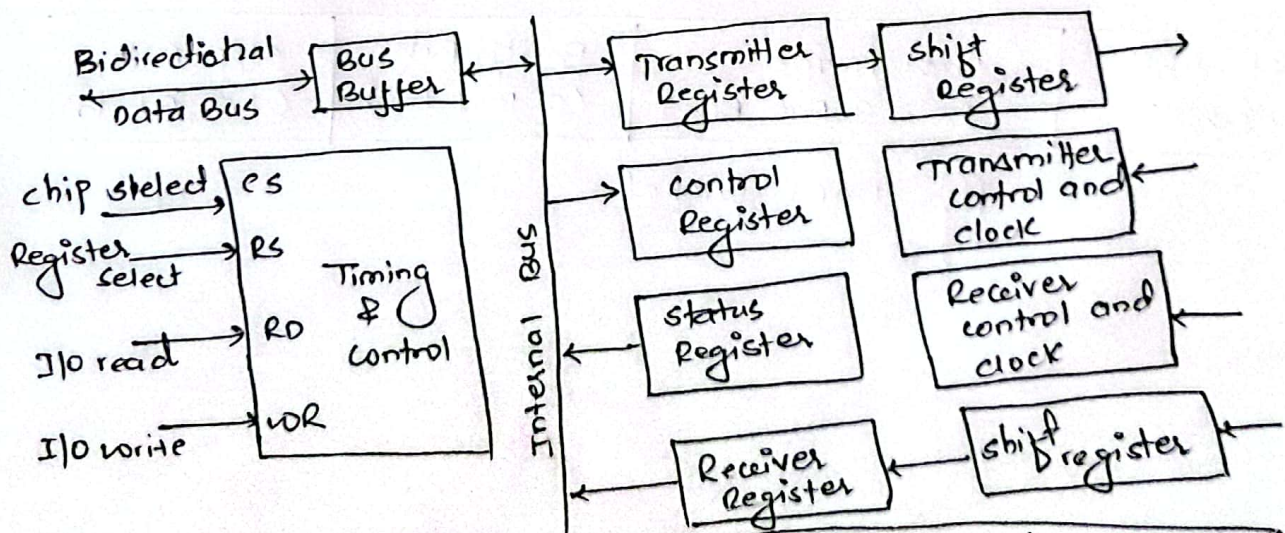
- ↳ It can handle I/O transfer with multiple devices at one time.

## 2.5. UART

- stands for Universal Asynchronous Receiver Transmitter
- In asynchronous serial transmission, binary information is sent only when it is available and the line remains ~~idle~~ idle, when there is no information to be transmitted.
- Both sender and receiver have their own clocks and these clocks work independent to each other.
- The Transmission process as follows:
  - ⊗ when a character is not being sent, the line is kept in the 1-state (Idle state)
  - ⊗ The initiation of a character transmission is detected from the start bit, which is always 0.
  - ⊗ The character bits always follow the start bit.
  - ⊗ After the last bit of the character is transmitted, a stop bit(s) is detected, which is always '1'.



↳ UART works both as transmitter and Receiver



| CS | RS | operation | Register selected.   |
|----|----|-----------|----------------------|
| 0  | X  | X         | None                 |
| 1  | 0  | WR        | Transmitter Register |
| 1  | 1  | WR        | Control Register     |
| 1  | 0  | RD        | Receiver Register    |
| 1  | 1  | RD        | Status Register.     |

## UART INTERFACE

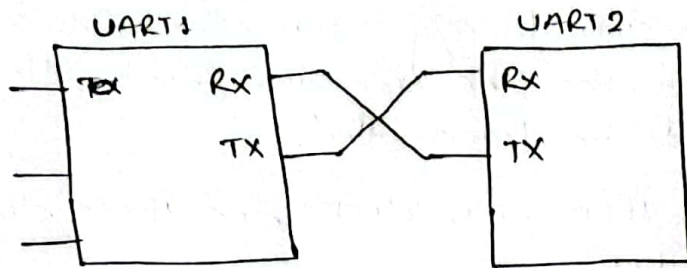


fig: Two UARTs directly communication with each other.

The two signals of each UART device are named:

- Transmitter (TX)
- Receiver (RX)

The main purpose of a transmitter and receiver line for each device is to transmit and receive serial data intended for serial communication.

### Data transmission:

In UART, the mode of transmission is in the form of a packet. The piece that connects the transmitter and receiver includes the creation of serial packets and controls those physical hardware lines. A packet consists of a start bit, data frame, a parity bit, and stop bits.

|                      |                                  |                             |                            |
|----------------------|----------------------------------|-----------------------------|----------------------------|
| Start bit<br>(1-bit) | Data frame<br>(5 to 9 Data Bits) | Parity Bits<br>(0 to 1 bit) | Stop Bits<br>(1 or 2 bits) |
|----------------------|----------------------------------|-----------------------------|----------------------------|

fig: UART packet.

## 8.6. USB and HDMI Standards

### #. USB (Universal Serial Bus)

Universal Serial Bus (USB) is an industry-standard that establishes specifications for connectors, cables and protocols for communication, connection, and power supply between personal sup computers and their peripheral devices.

~~There is~~ The universal serial Bus was designed to simplify and improve the interface between personal computers and peripheral devices when compared with previously existing standard.

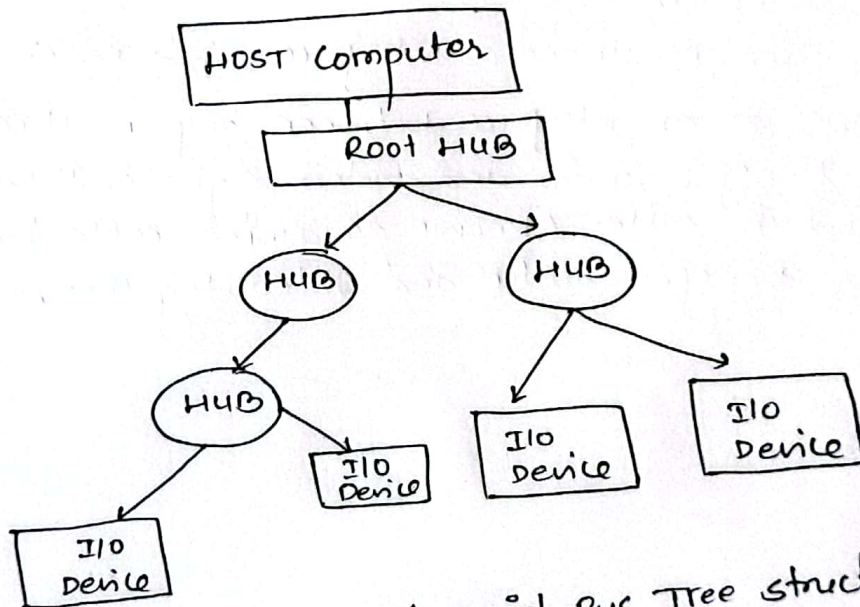


fig: universal serial Bus Tree structure.

↳ The major advantages of USB are:

- i) The USB interface is self-configuring. This means that the user need not adjust settings on the device and interface for speed or data format, or configure interrupts, input/output addresses or direct memory access channels.
- ii) The USB interface is hot pluggable or plug and plays, meaning devices can be exchanged without rebooting the host computer. Small devices can be powered directly from the USB interface thus removing extra power supply cables.
- iii) The USB interface defines protocols for improving reliability over previous interfaces and recovery from common errors.
- iv) Installation of a device rep. relying on the USB standard minimal operation action is required.

## #. Disadvantages of USB-

1. USB cables are limited in length
2. USB has strict 'tree' topology and "master-slave" protocol for addressing peripheral devices. Peripheral devices cannot interact with out another ~~ex~~ except via the host, and two hosts cannot communicate over their USB ports directly.
3. Some very high speed peripheral devices require standard sustained speeds not available in the USB standard.

## #. HDMI standard.

- ↳ High Definition Multimedia Interface, is a ~~port~~.
- ↳ HDMI works as an interface between any multimedia devices to transfer audio and video signals. It is mainly a port in which cables (known as HDMI cables) are connected to transfer audio and video together.

↳